---

1. Question 1: Printing the Page Table of a Process

1.1 Implementation Details

The goal is to inspect the two-level paging hierarchy of the calling process in xv6 (x86), where user virtual addresses range from `0` to `KERNBASE` (`0x80000000`).

- **`vm.c` (`vmprint`)**:
  - Implemented `int vmprint(pde_t *pgdir)` which iterates through page directory entries up to `PDX(KERNBASE)` (indexes `0` to `511`), ignoring the kernel mapping above `KERNBASE`.
  - For every present page directory entry (`pgdir[i] & PTE_P`), it increments `pt_pages` and accesses the page table at virtual address `P2V(PTE_ADDR(pgdir[i]))`.
  - Iterates through all 1024 PTEs in each page table. For present entries (`pte & PTE_P`), it computes:
    - Virtual address: `PGADDR(i, j, 0)`
    - Physical frame address: `PTE_ADDR(pte)`
    - Flags: Present (`P`), Writable (`W`), User (`U`)
  - Increments `user_pages` and prints each mapped user page, followed by total valid user pages and page-table pages.
- **System Call Hookup**:
  - Registered system call `SYS_vmprint` (number 22) across `syscall.h`, `syscall.c`, `user.h`, and `usys.S`.
  - Added wrapper `sys_vmprint()` in `sysproc.c` invoking `vmprint(myproc()->pgdir)`.
  - Declared `vmprint(pde_t*)` in `defs.h`.
- **User Program (`pttest.c`)**:
  - Calls `vmprint()` at startup, executes `sbrk(4 * 4096)`, touches each allocated page, and calls `vmprint()` again.

2. Question 2: Handling Page Faults (Lazy Allocation)

2.1 Implementation Details

Physical memory allocation is deferred until a page is first accessed:

- **`sysproc.c` (`sys_sbrk`)**: When n > 0, `sys_sbrk()` only increases `myproc()->sz += n` without calling `allocuvm()`. For n < 0, `growproc(n)` deallocates memory as usual.
- **`vm.c` (`copyuvm` & `deallocuvm`)**:

    - In `copyuvm()`, replaced panics on missing PTEs or non-present pages with `continue;`, allowing `fork()` to copy sparsely allocated address spaces.
    - `deallocuvm()` already ignores unmapped pages (`else if((*pte & PTE_P) != 0)`), preventing crashes when unallocated heap space is freed at process exit.
    - Made `walkpgdir` and `mappages` non-static and declared them in `defs.h`.
- **`trap.c` (`T_PGFLT`)**:
    - Traps page fault interrupt `T_PGFLT` (trap 14). Reads fault address via `rcr2()`.
    - If `va < myproc()->sz` and the page is not yet mapped, it allocates a physical page using `kalloc()`, clears it with `memset()`, maps it using `mappages(..., PTE_W | PTE_U)`, increments `myproc()->faults`, and resumes execution.
    - If `va >= myproc()->sz` (out-of-bounds user access), prints PID, process name, and fault address, and kills the process (`myproc()->killed = 1`).
- **Fault Tracking & `getfaults()`**:
    - Added `uint faults;` to `struct proc` in `proc.h`, initialized to 0 in `allocproc()` and reset to 0 upon `exec()`.
    - Added system call `getfaults()` (number 23) to retrieve the fault count.
- **User Program (`faulttest.c`)**:
    - Tests that `sbrk(32 * 4096)` triggers 0 faults initially.
    - Writes to 5 distinct pages and confirms `getfaults()` returns exactly 5.
    - Forks a child that accesses `p + 33 * 4096` (above `sz`); confirms the child is killed while the parent remains unaffected.

3. Question 3: Page Replacement Algorithms (FIFO & LRU)

3.1 Implementation Details

Implemented in user-space program `pagerepl.c`:

- **FIFO**: Uses a circular pointer (`ptr = (ptr + 1) % num_frames`) to replace the oldest loaded frame.
- **LRU**: Tracks the reference timestamp of each frame in `last_used[]`. On a fault, it selects the frame index with \min(\text{last_used}).
- **Modes**: Supports automated assignment testing, custom CLI arguments (`pagerepl <frames> [p1 p2 ...]`), and interactive input (`pagerepl -i`).

3.3 Comparative Discussion

1. **FIFO vs. LRU**: LRU consistently outperforms FIFO across all frame allocations (12 vs. 15 faults at 3 frames, 8 vs. 10 at 4 frames, and 7 vs. 9 at 5 frames). LRU leverages temporal locality by evicting pages not used for the longest period, whereas FIFO blindly evicts the oldest page even if it is currently in active reuse.

2. **Impact of Frame Count**: Increasing frame capacity from 3 to 5 reduces page faults significantly (from 15 to 9 for FIFO, and from 12 to 7 for LRU), monotonically increasing the hit ratio from 25% to 55% (FIFO) and 40% to 65% (LRU).
